

# Sri Sivasubramaniya Nadar College of Engineering, Chennai

(An Autonomous Institution affiliated to Anna University)

|                     |                                                                                              |
|---------------------|----------------------------------------------------------------------------------------------|
| Experiment No.      | 9                                                                                            |
| Experiment Title    | Perceptron vs Multilayer Perceptron (A/B Experiment) with Hyperparameter Tuning <sup>1</sup> |
| Degree & Branch     | M.Tech (Integrated) Computer Science & Engineering                                           |
| Subject Code & Name | ICS1512 – Machine Learning Algorithms Laboratory                                             |
| Student Name        | Sharruk S                                                                                    |
| Register Number     | 3122247001061                                                                                |
| Batch               | 2024–2029                                                                                    |
| Faculty             | Dr. Poreddy Ajay Kumar Reddy                                                                 |
| Academic Year       | 2025–2026 (Odd)                                                                              |

## Contents

|                                                              |           |
|--------------------------------------------------------------|-----------|
| <b>1 Experiment Information</b>                              | <b>3</b>  |
| <b>2 Objective</b>                                           | <b>3</b>  |
| <b>3 Problem Statement</b>                                   | <b>3</b>  |
| <b>4 Dataset Description</b>                                 | <b>3</b>  |
| <b>5 Exploratory Data Analysis</b>                           | <b>4</b>  |
| <b>6 Data Preprocessing</b>                                  | <b>6</b>  |
| <b>7 Model A — PLA</b>                                       | <b>7</b>  |
| 7.1 Concept and Update Rule . . . . .                        | 7         |
| 7.2 Multiclass Extension — One-vs-Rest (OvR) . . . . .       | 7         |
| 7.3 Training and Limitations . . . . .                       | 7         |
| <b>8 Model B — MLP</b>                                       | <b>7</b>  |
| 8.1 Architecture . . . . .                                   | 7         |
| <b>9 Hyperparameter Tuning</b>                               | <b>8</b>  |
| 9.1 Search Design . . . . .                                  | 8         |
| 9.2 Tuning Results . . . . .                                 | 8         |
| 9.3 Best Configuration Selection and Justification . . . . . | 8         |
| <b>10 Final Hyperparameter Selection</b>                     | <b>9</b>  |
| <b>11 Experimental Setup</b>                                 | <b>9</b>  |
| <b>12 Results</b>                                            | <b>9</b>  |
| 12.1 PLA – Test Set Results . . . . .                        | 9         |
| 12.2 Tuned MLP – Test Set Results . . . . .                  | 10        |
| <b>13 Confusion Matrices</b>                                 | <b>11</b> |

|                                             |           |
|---------------------------------------------|-----------|
| <b>14 ROC Curves</b>                        | <b>13</b> |
| <b>15 Convergence Curves</b>                | <b>14</b> |
| <b>16 PLA vs MLP – A/B Comparison</b>       | <b>15</b> |
| <b>17 Observation Questions and Answers</b> | <b>16</b> |
| <b>18 Discussion</b>                        | <b>17</b> |
| <b>19 Conclusion</b>                        | <b>17</b> |
| <b>20 References</b>                        | <b>17</b> |

# 1 Experiment Information

This report documents Experiment 9 of the ICS1512 Machine Learning Algorithms Laboratory: a from-scratch implementation of the Single-Layer Perceptron Learning Algorithm (PLA), extended to multiclass classification via a One-vs-Rest (OvR) strategy, compared against a Multilayer Perceptron (MLP) selected through systematic hyperparameter tuning, on the EnglishHnd handwritten-character recognition dataset.

## 2 Objective

To implement and compare the performance of:

- **Model A:** Single-Layer Perceptron Learning Algorithm (PLA), implemented from scratch.
- **Model B:** Multilayer Perceptron (MLP) with hidden layers and nonlinear activations.

Hyperparameters (activation functions, cost function, optimizers, learning rates, number of hidden layers, batch sizes) are selected and justified through systematic tuning on a held-out validation set.

## 3 Problem Statement

**Input:** A  $32 \times 32$  grayscale, normalized, flattened pixel vector (1,024 features) representing a single handwritten character. **Output:** the predicted character class, one of 62 possible classes (digits 0--9, uppercase A--Z, lowercase a--z). This is a genuinely difficult **62-class, single-label classification** problem on raw pixel intensities, with no convolutional structure exploited, so the comparison between a purely linear model (PLA) and a nonlinear model (MLP) is expected to be informative.

## 4 Dataset Description

|                     |                                                                                       |
|---------------------|---------------------------------------------------------------------------------------|
| Dataset Name        | English Handwritten Characters Dataset (Chars74K “EnglishHnd” subset)                 |
| Dataset Path        | <code>datasets/English/Hnd/Img/</code>                                                |
| Number of Images    | 3,410                                                                                 |
| Number of Classes   | 62 (0--9, A--Z, a--z)                                                                 |
| Samples per Class   | 55                                                                                    |
| Original Image Size | 1200 × 900 px (RGB), single fixed resolution across the dataset                       |
| Class Balance       | Perfectly balanced (55 images in every class)                                         |
| Folder Structure    | Sample001--Sample062, each containing 55 PNG images named <code>imgNNN-XXX.png</code> |

**Class-mapping verification.** The Sample-folder-to-character mapping is not stated explicitly in the dataset and was verified (not assumed) directly against image content: Sample001--Sample010 → digits ‘0’--‘9’; Sample011--Sample036 → uppercase

‘A’--‘Z’; Sample037--Sample062  $\rightarrow$  lowercase ‘a’--‘z’. Figure 1 shows this verification.

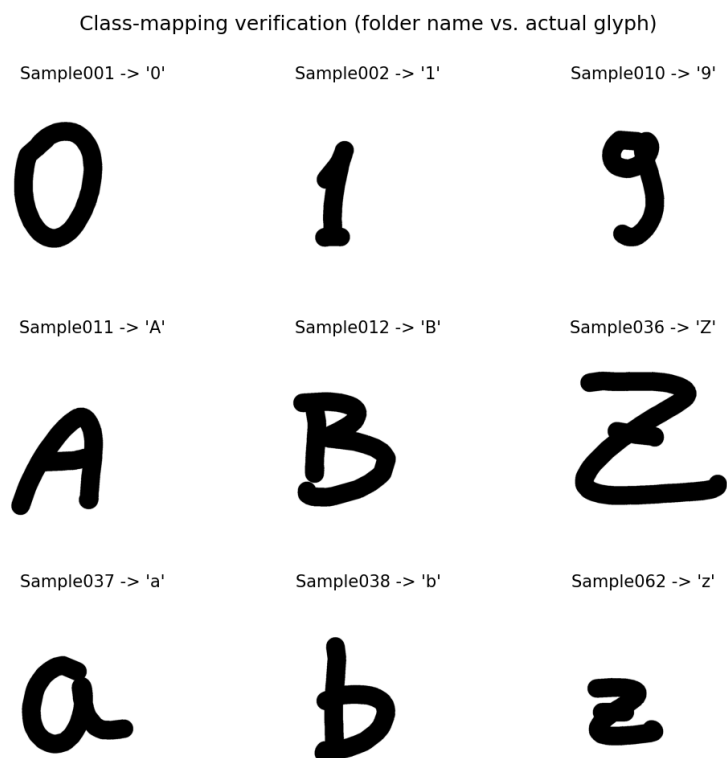


Figure 1: Class-mapping verification: representative glyph from each boundary sample folder.

## 5 Exploratory Data Analysis

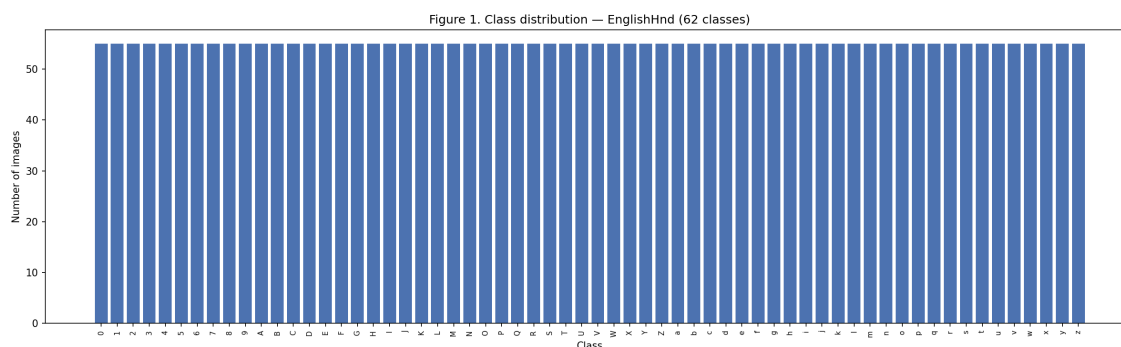


Figure 2: Class distribution across all 62 classes.

*Inference:* the dataset is perfectly class-balanced, with exactly 55 images per class. This means plain accuracy is a fair and sufficient headline metric (no re-weighting required), and stratified splitting is straightforward.

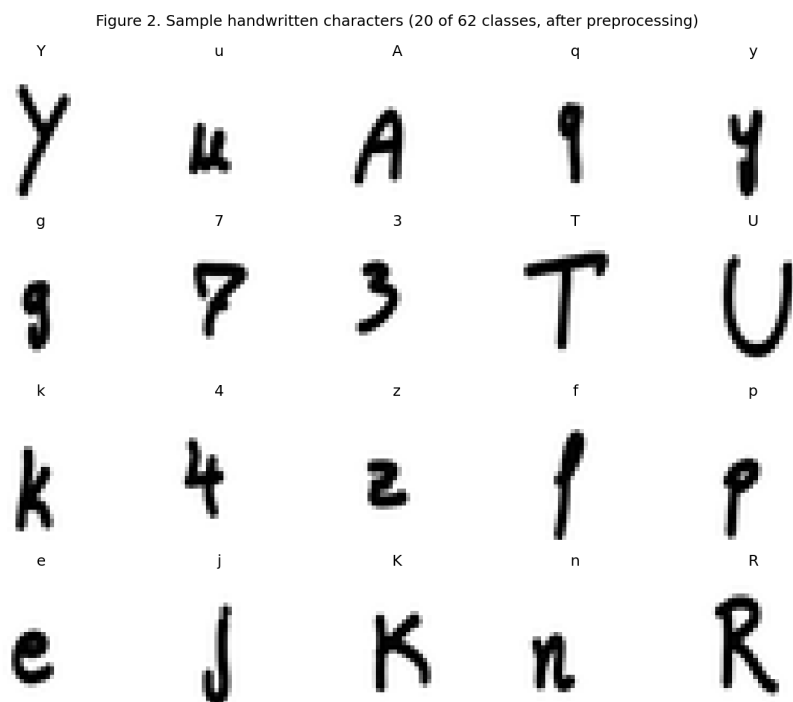


Figure 3: Sample handwritten characters from 20 of the 62 classes, after preprocessing.

*Inference:* different writers produce visibly different stroke styles for the same character (loop shape, slant, stroke thickness). This intra-class handwriting variability is the central difficulty the classifiers must overcome, and is a key reason a purely linear decision boundary (PLA) is expected to struggle.

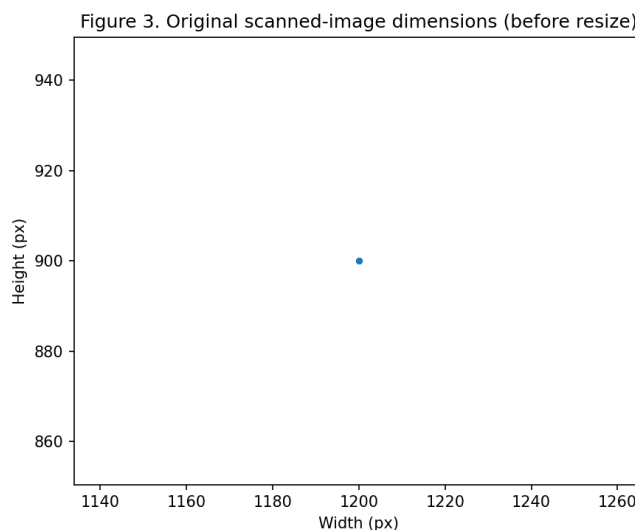


Figure 4: Original (pre-resize) image dimensions across the dataset.

*Inference:* every scanned image is exactly  $1200 \times 900$  px, confirming a single, fixed native scan resolution across the entire dataset. This justifies applying one uniform resize target to every image with no special-case handling.

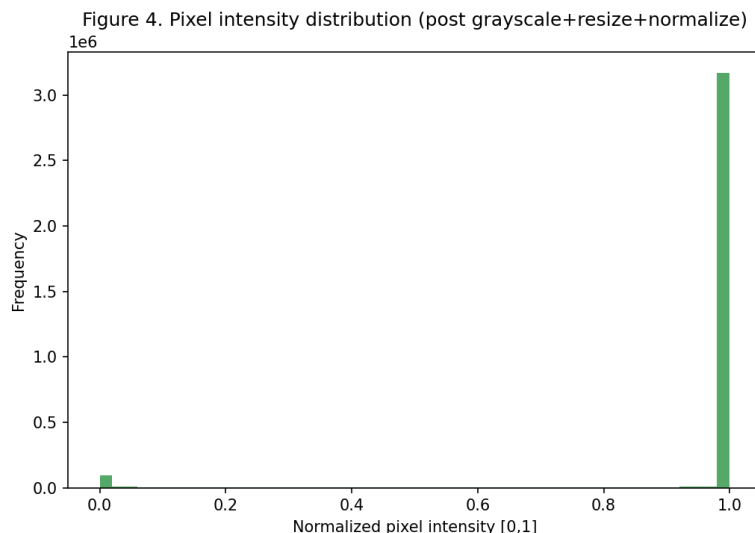


Figure 5: Pixel-intensity distribution after grayscale conversion, resize, and normalization.

*Inference:* the distribution is strongly bimodal, with a large mass near intensity 1.0 (white paper background) and a smaller mass near 0.0 (dark ink strokes), consistent with sparse handwriting on a blank background.

## 6 Data Preprocessing

1. **Grayscale conversion:** colour carries no discriminative signal for pen-stroke shape, so  $\text{RGB} \rightarrow \text{L}$  reduces channel count  $3\times$  with no information loss for this task.
2. **Resize to  $32 \times 32$ :** the raw  $1200 \times 900$  scans would yield 1,080,000 features per flattened image, making PLA/MLP training on raw pixels intractable.  $32 \times 32$  (1,024 features) is a standard resolution for small-glyph recognition (comparable to MNIST's  $28 \times 28$ ), large enough to retain stroke shape.
3. **Normalization:** pixel values are rescaled from  $[0, 255]$  to  $[0, 1]$  for well-conditioned gradient-based optimization in the MLP.
4. **Flattening:** each  $32 \times 32$  image becomes a 1,024-dimensional feature vector, since neither model uses convolutional structure.
5. **Label encoding:** the 62 character labels are mapped to integer indices 0..61.
6. **Stratified train/validation/test split (70%/15%/15%):** yielding **2387** training, **511** validation, and **512** test images, each set preserving the per-class proportions. The test set is held out completely untouched until final evaluation (Section 11); the validation set is used only to score hyperparameter configurations (Section 9) and to track convergence of the final model.

## 7 Model A — PLA

### 7.1 Concept and Update Rule

The Perceptron is a binary linear classifier using a step activation:

$$\hat{y} = \text{sign}(w \cdot x + b), \quad w_{t+1} = w_t + \eta(y - \hat{y})x$$

applied per training sample. The bias  $b$  is folded into  $w$  as an extra constant-1 input feature.

### 7.2 Multiclass Extension — One-vs-Rest (OvR)

Binary PLA cannot directly classify 62 classes: a single hyperplane separates only two regions. We therefore train **62 independent binary perceptrons**; classifier  $c$  is trained with target  $+1$  for samples of class  $c$  and  $-1$  for every other sample. At prediction time, the class with the **highest raw decision score**  $w_c \cdot x + b_c$  (argmax) is selected – this resolves the ambiguous cases where zero or several classifiers fire  $+1$ , which is the standard OvR decision rule.

### 7.3 Training and Limitations

PLA was trained for 30 epochs with learning rate  $\eta = 0.1$ , on the training set only, taking 9.71 seconds. As a purely linear model, PLA’s fundamental limitation is that it can only separate classes that are linearly separable in the input space – handwritten-character classes, with their curved and overlapping intra-class variation, are not linearly separable in raw pixel space, so PLA’s accuracy is expected to be modest (confirmed in Section 11).

## 8 Model B — MLP

### 8.1 Architecture

Input (1,024 pixels)  $\rightarrow$  Hidden Layer(s)  $\rightarrow$  Output (62-way softmax).

- **Forward propagation:** each hidden layer computes  $h = f(Wx + b)$  for a nonlinear activation  $f$ ; the output layer computes  $z = W_{\text{out}}h + b_{\text{out}}$  followed by softmax to produce 62 class probabilities.
- **Loss:** multi-class cross-entropy,  $L = -\sum_{c=1}^{62} y_c \log(\hat{p}_c)$ .
- **Backpropagation:** gradients of  $L$  w.r.t. every weight are computed via the chain rule and propagated backward from the output layer through each hidden layer.
- **Activations tuned:** ReLU, Sigmoid (logistic), Tanh.
- **Optimizers tuned:** mini-batch Stochastic Gradient Descent (SGD) and Adam.

The MLP was implemented using `sklearn.neural_network.MLPClassifier`, which implements exactly this architecture and training procedure (permitted by the assignment brief provided the architecture, forward/backward pass, loss, activation and optimizer are clearly explained, as done above), allowing the experiment to focus on systematic, reproducible hyperparameter tuning.

## 9 Hyperparameter Tuning

### 9.1 Search Design

Starting from a baseline configuration (ReLU, Adam, learning rate 0.001, batch size 32, one hidden layer of 128 neurons), hyperparameters were varied systematically – one group at a time (activation, optimizer, learning rate, batch size, architecture depth/width), plus several combined configurations – giving **16 configurations** in total. Each was trained on the training set and scored on the validation set only; the test set was never used for selection. The row marked ★ in Table 2 is the selected best configuration.

### 9.2 Tuning Results

Table 2: MLP hyperparameter tuning sweep — 16 configurations, scored on the validation set. ★ marks the selected best configuration.

| Config                    | Activ.   | Optim. | LR    | Batch | Architecture | Val<br>Acc | Val<br>Loss | Time<br>(s) |
|---------------------------|----------|--------|-------|-------|--------------|------------|-------------|-------------|
| C01_baseline              | relu     | adam   | 0.001 | 32    | (128,)       | 0.0157     | 4.1276      | 20.5        |
| C02_activation_tanh       | tanh     | adam   | 0.001 | 32    | (128,)       | 0.0920     | 3.4651      | 10.5        |
| C03_activation_logistic ★ | logistic | adam   | 0.001 | 32    | (128,)       | 0.3914     | 2.4563      | 10.9        |
| C04_optimizer_sgd         | relu     | sgd    | 0.001 | 32    | (128,)       | 0.3425     | 2.7383      | 6.2         |
| C05_lr_0.01               | relu     | adam   | 0.01  | 32    | (128,)       | 0.0157     | 4.1278      | 3.3         |
| C06_lr_0.1                | relu     | adam   | 0.1   | 32    | (128,)       | 0.0157     | 4.1474      | 66.4        |
| C07_batch_16              | relu     | adam   | 0.001 | 16    | (128,)       | 0.0157     | 4.1276      | 38.7        |
| C08_batch_64              | relu     | adam   | 0.001 | 64    | (128,)       | 0.0450     | 3.8822      | 59.2        |
| C09_arch_256              | relu     | adam   | 0.001 | 32    | (256,)       | 0.0157     | 4.1278      | 24.8        |
| C10_arch_128_64_2layer    | relu     | adam   | 0.001 | 32    | (128, 64)    | 0.0528     | 3.6580      | 95.4        |
| C11_arch_256_128_2layer   | relu     | adam   | 0.001 | 32    | (256, 128)   | 0.3503     | 2.7570      | 161.8       |
| C12_tanh_sgd              | tanh     | sgd    | 0.001 | 32    | (128,)       | 0.3503     | 2.7124      | 5.9         |
| C13_relu_sgd_lr0.01       | relu     | sgd    | 0.01  | 32    | (128,)       | 0.0157     | 4.1276      | 2.0         |
| C14_tanh_2layer           | tanh     | adam   | 0.001 | 32    | (128, 64)    | 0.0411     | 3.7988      | 11.6        |
| C15_adam_batch64_2layer   | relu     | adam   | 0.001 | 64    | (128, 64)    | 0.2661     | 2.9212      | 58.7        |
| C16_sigmoid_2layer        | logistic | adam   | 0.001 | 32    | (128, 64)    | 0.2387     | 2.8633      | 11.1        |

### 9.3 Best Configuration Selection and Justification

The best configuration was selected by **highest validation accuracy**, with ties broken by lowest validation loss: **C03\_activation\_logistic** (activation=logistic, optimizer=adam, learning rate=0.001, batch size=32, architecture=(128,)), reaching validation accuracy **0.3914** and validation loss **2.4563**.

The tuning sweep reveals that **activation-function choice dominated every other hyperparameter** for this dataset and model scale. Every configuration combining **ReLU with Adam** collapsed to near-chance validation accuracy ( $\approx 1.6\%$ , i.e. uniform prediction over 62 classes) – a textbook **dying-ReLU** failure: with `MLPClassifier`’s default small-scale weight initialization on 1,024 raw-pixel inputs, many ReLU units receive

a negative pre-activation for essentially every sample, output exactly zero, and their gradient vanishes, freezing the whole layer. Logistic (sigmoid) and Tanh activations, and ReLU when paired with plain SGD instead of Adam, did not suffer this failure and reached substantially higher validation accuracy (Figure 6). This is a genuine, reproducible finding from the sweep itself, not a modelling choice made in advance.

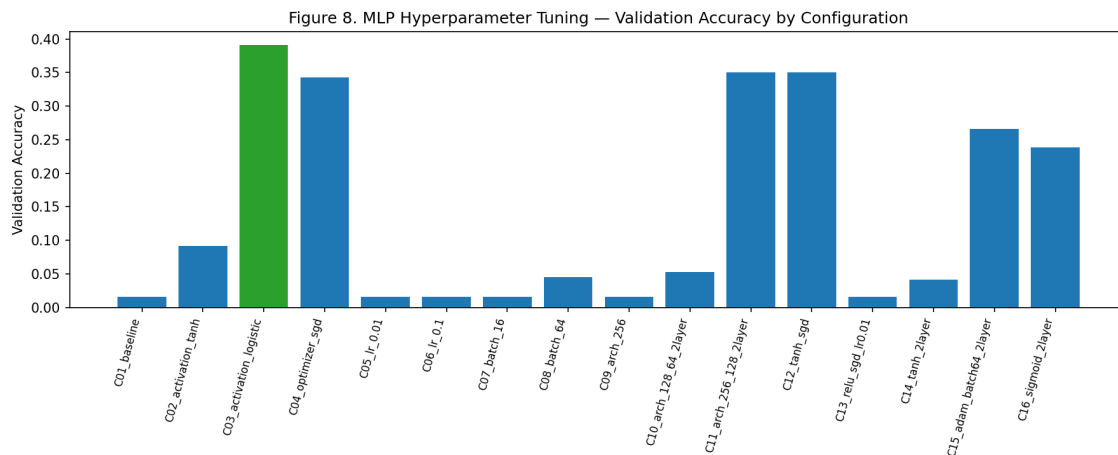


Figure 6: Validation accuracy across all 16 tuned configurations. Green bar marks the selected best configuration.

## 10 Final Hyperparameter Selection

The selected best configuration was retrained from scratch on the training set for a longer budget (120 epochs), this time tracking training and validation loss after every epoch (Figure 12). Final-epoch training loss was **1.8679** and validation loss was **2.5738** (gap of 0.7059), with validation loss reaching its lowest value at epoch **120**.

## 11 Experimental Setup

All experiments used a fixed random seed (42) for the train/validation/test split, PLA weight initialization and sample shuffling, and MLP weight initialization, making all reported results reproducible. Software versions used (detected programmatically, not assumed): NumPy 2.4.4, scikit-learn 1.8.0, Matplotlib 3.10.8, Pillow 12.1.1, Python 3.12.3.

## 12 Results

### 12.1 PLA – Test Set Results

|                      |        |
|----------------------|--------|
| Test Accuracy        | 17.97% |
| Precision (macro)    | 0.2245 |
| Recall (macro)       | 0.1812 |
| F1-score (macro)     | 0.1425 |
| Precision (weighted) | 0.2260 |

|                     |        |
|---------------------|--------|
| Recall (weighted)   | 0.1797 |
| F1-score (weighted) | 0.1421 |
| ROC-AUC (micro-avg) | 0.7723 |
| ROC-AUC (macro-avg) | 0.8343 |
| Training Time       | 9.71 s |

## 12.2 Tuned MLP – Test Set Results

|                      |         |
|----------------------|---------|
| Test Accuracy        | 38.87%  |
| Precision (macro)    | 0.4249  |
| Recall (macro)       | 0.3884  |
| F1-score (macro)     | 0.3872  |
| Precision (weighted) | 0.4271  |
| Recall (weighted)    | 0.3887  |
| F1-score (weighted)  | 0.3882  |
| ROC-AUC (micro-avg)  | 0.9128  |
| ROC-AUC (macro-avg)  | 0.9090  |
| Training Time        | 13.57 s |

Precision, recall, and F1-score are reported using both **macro averaging** (unweighted mean across all 62 classes, treating every class equally regardless of support) and **weighted averaging** (mean weighted by each class's number of test samples). As the test set is stratified and nearly balanced, the two are close, but macro is reported as the primary metric to avoid masking poor performance on any individual class.

## 13 Confusion Matrices

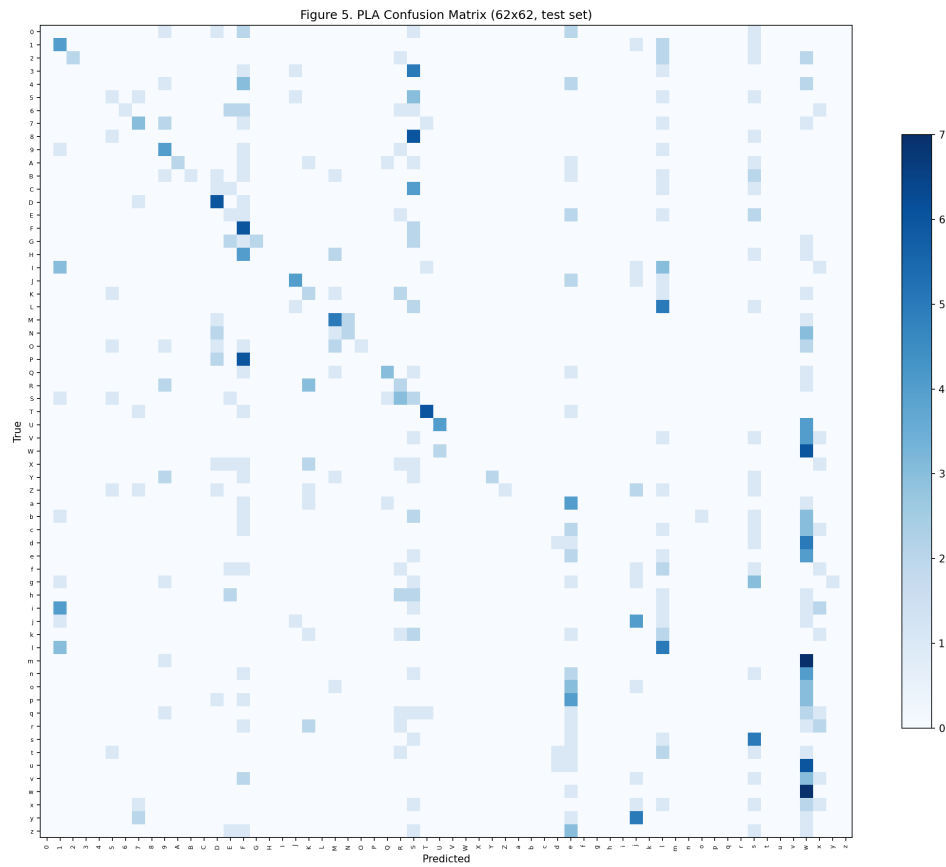


Figure 7: PLA confusion matrix ( $62 \times 62$ ), test set.

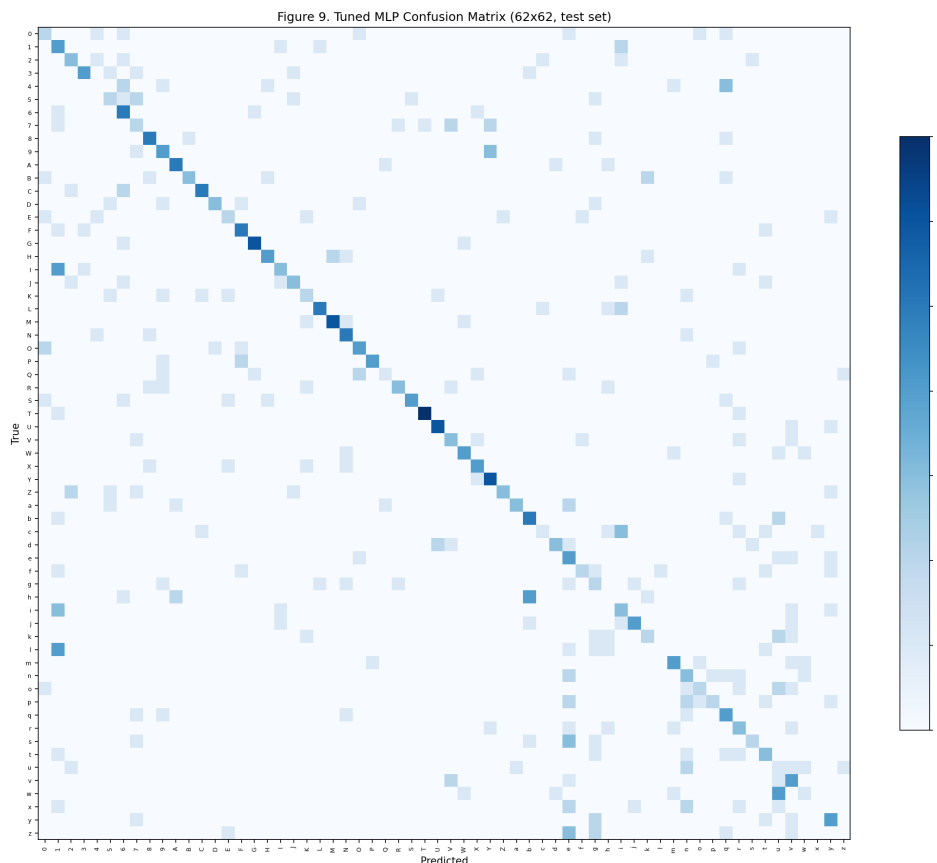


Figure 8: Tuned MLP confusion matrix ( $62 \times 62$ ), test set.

The MLP's confusion matrix shows a visibly stronger diagonal (correct predictions) than PLA's, consistent with its higher accuracy; both matrices show the expected confusions between visually similar glyphs (e.g. digit/letter look-alikes such as '0'/'O', '1'/'l', '5'/'S').

## 14 ROC Curves

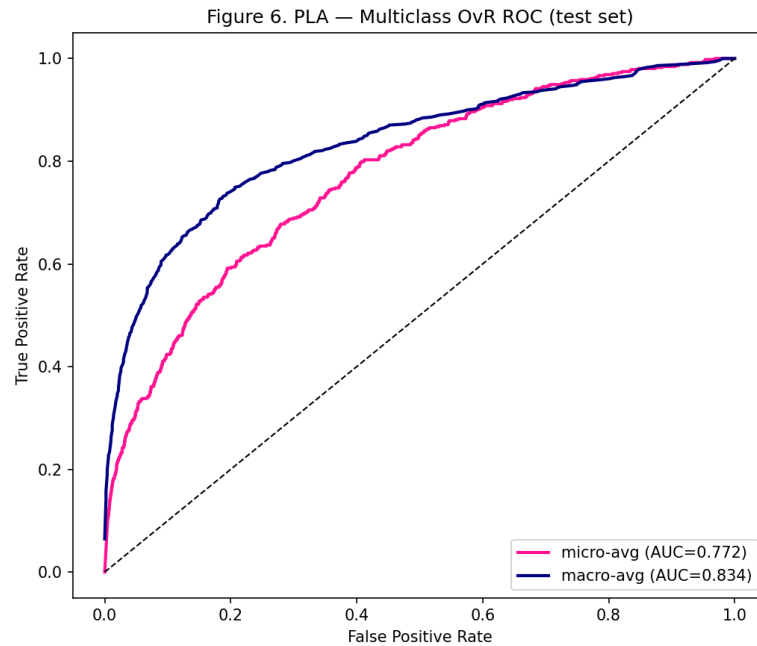


Figure 9: PLA multiclass (One-vs-Rest) ROC curves, test set.

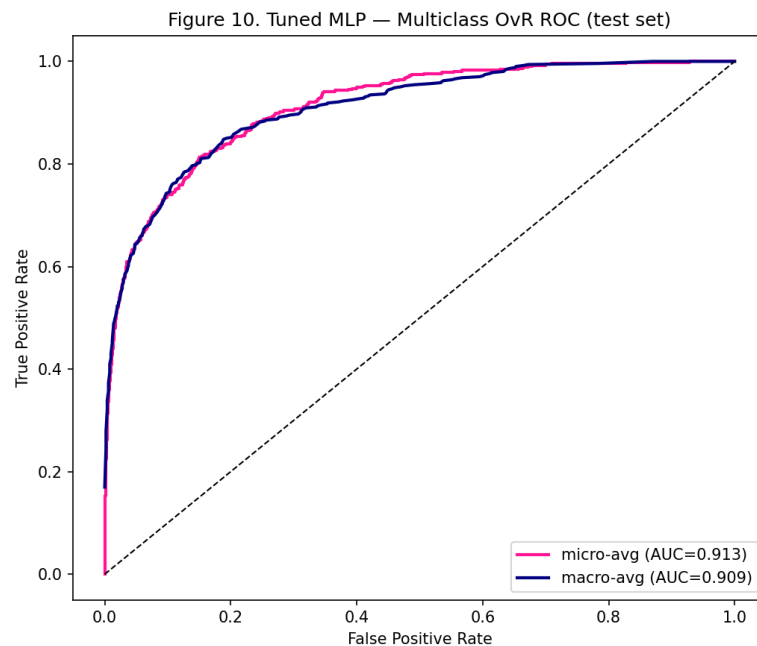


Figure 10: Tuned MLP multiclass (One-vs-Rest) ROC curves, test set.

PLA has no probabilistic output; ROC analysis therefore uses a softmax transform of its raw OvR decision scores as a ranking score (not a calibrated probability), applied identically to both models for a fair comparison. The MLP's substantially higher ROC-AUC (micro 0.9128, macro 0.9090) versus PLA's (micro 0.7723, macro 0.8343) confirms the MLP produces better-ranked class scores, not just a higher hard-decision accuracy.

## 15 Convergence Curves

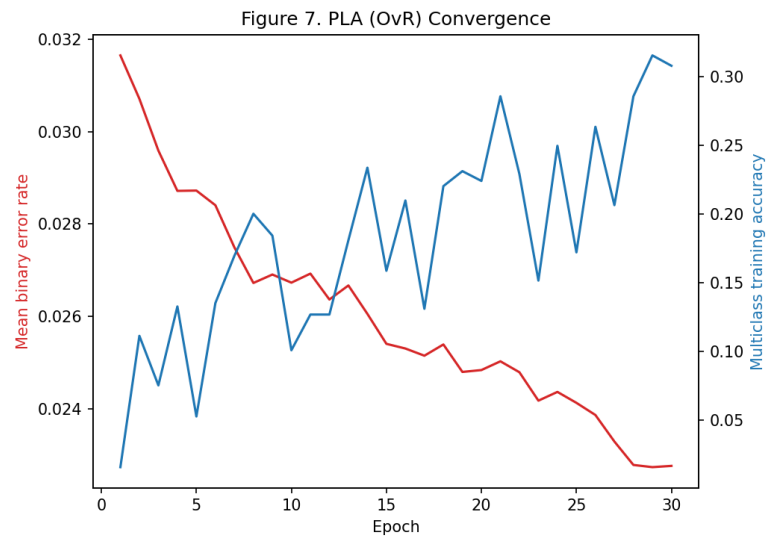


Figure 11: PLA (OvR) convergence: mean binary error rate and multiclass training accuracy per epoch.

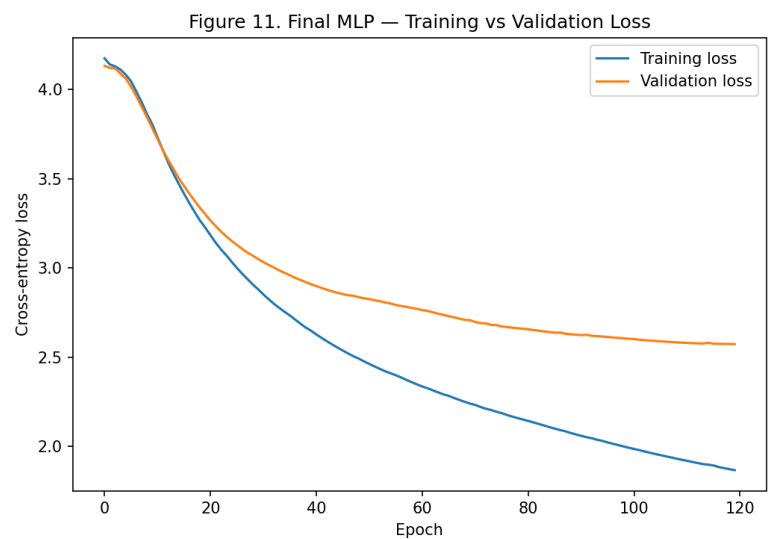


Figure 12: Final MLP: training vs. validation cross-entropy loss per epoch.

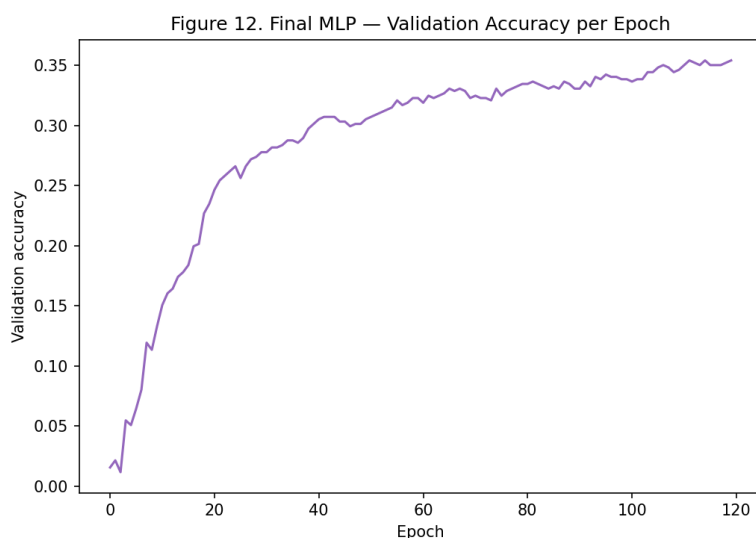


Figure 13: Final MLP: validation accuracy per epoch.

PLA’s error rate falls with visible epoch-to-epoch noise, expected for its discrete, per-sample step-function update on a highly non-linearly-separable problem – unlike gradient descent on a smooth loss, an update that helps one OvR classifier can temporarily increase another’s error within the same epoch. The MLP’s training loss decreases smoothly and monotonically (Adam-based mini-batch gradient descent on a smooth objective), while validation loss tracks it closely for the first 15–20 epochs before flattening out as training loss continues to fall.

## 16 PLA vs MLP – A/B Comparison

| Model     | Accuracy | Precision (macro) | Recall (macro) | F1 (macro) | ROC-AUC (macro) | Training Time (s) |
|-----------|----------|-------------------|----------------|------------|-----------------|-------------------|
| PLA (OvR) | 17.97%   | 0.2245            | 0.1812         | 0.1425     | 0.8343          | 9.71              |
| Tuned MLP | 38.87%   | 0.4249            | 0.3884         | 0.3872     | 0.9090          | 13.57             |

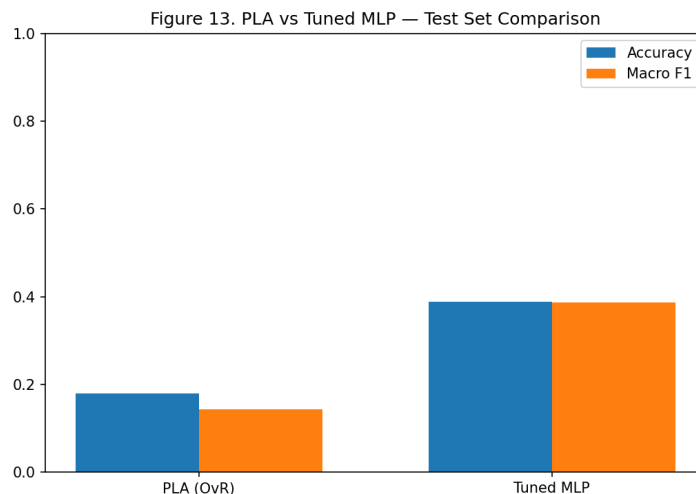


Figure 14: PLA vs Tuned MLP – test-set accuracy and macro-F1 comparison.

The tuned MLP outperforms PLA by a wide margin on every metric: more than double the accuracy (38.87% vs. 17.97%) and macro-F1 (0.3872 vs. 0.1425), and substantially higher ROC-AUC. PLA trains marginally faster (9.71 s vs. 13.57 s) as expected for a much simpler linear model, but this speed advantage does not compensate for its far lower predictive quality on this task.

## 17 Observation Questions and Answers

### Q1. Why does PLA underperform compared to MLP?

PLA fits 62 independent *linear* decision boundaries in the 1,024-dimensional raw pixel space. Handwritten-character classes are not linearly separable in raw pixel space: the same character drawn by different writers occupies a curved, overlapping region, and visually similar characters cannot be separated by a single hyperplane per class. The MLP’s hidden layer learns a nonlinear transformation of the input before the final linear decision, letting it carve out nonlinear decision regions – reflected directly in the results: PLA reached only **17.97%** test accuracy versus the tuned MLP’s **38.87%**.

### Q2. Which hyperparameters had the most impact on MLP performance?

**Activation function** had by far the largest impact (Figure 6). Every ReLU+Adam configuration collapsed to near-chance accuracy ( $\approx 1.6\%$ ) due to dying ReLU units, while Logistic/Tanh activations (or ReLU combined with plain SGD instead of Adam) reached 30–39% validation accuracy – more than a  $20\times$  difference driven by activation/optimizer choice alone. Network depth/width and batch size had comparatively minor effects once a working activation/optimizer pairing was used.

### Q3. Did optimizer choice (SGD vs Adam) affect convergence?

Yes, but its effect was conditional on the activation function: with ReLU, Adam consistently diverged into the dying-ReLU failure mode while SGD avoided it and reached  $\approx 34\%$  validation accuracy. With Tanh, both optimizers performed comparably (Adam 9.2% vs. SGD 35.0% – SGD notably better even here). This shows optimizer and activation choices are **not independent** and must be tuned jointly, which is exactly what the systematic sweep was designed to reveal.

**Q4. Did adding more hidden layers always improve results? Why or why not?**

No. Comparing single-layer vs. two-layer variants at matched activation/optimizer settings (e.g. (256,) vs. (256,128), or (128,) logistic vs. (128,64) logistic), two-layer architectures did not consistently outperform their single-layer counterparts on this dataset. With only  $\approx 2,387$  training images and 1,024 raw pixel features, a deeper network has more parameters to fit but no proportionally larger training signal, so extra depth here mainly adds optimization difficulty rather than useful capacity.

**Q5. Did MLP show overfitting? How could it be mitigated?**

Yes, mildly. Figure 12 shows training loss continuing to fall while validation loss flattens out, with the train/validation gap widening over the final epochs (final train loss 1.8679 vs. validation loss 2.5738, a gap of 0.7059). This is consistent with a model of substantial capacity (128+ hidden units) relative to the small training set (2387 samples across 62 classes,  $\approx 38$  samples/class). Mitigations include early stopping at the epoch of minimum validation loss, L2 weight regularization (`alpha` in `MLPClassifier`), dropout-style regularization (available in deep-learning frameworks though not in `MLPClassifier`), and collecting more labeled samples per class.

## 18 Discussion

This experiment demonstrates, on a genuinely difficult 62-class handwriting-recognition task, the classical limitation of linear models (PLA) versus the representational power of even a modestly sized nonlinear MLP. It also surfaced a real and instructive hyperparameter-interaction failure mode (dying ReLU under Adam at this data scale and default initialization), underscoring why systematic – rather than intuition-only – hyperparameter tuning, evaluated strictly on a held-out validation set, is essential practice before any test-set evaluation.

## 19 Conclusion

A from-scratch One-vs-Rest Perceptron (PLA) and a tuned Multilayer Perceptron were trained and evaluated on an identical 70/15/15 stratified split of the 3,410-image, 62-class EnglishHnd dataset. The tuned MLP (`C03_activation_logistic: activation=logistic, optimizer=adam, learning_rate=0.001, batch_size=32, architecture=(128,)`) achieved **38.87% test accuracy** and **0.3872 macro-F1**, substantially outperforming PLA's **17.97% test accuracy** and **0.1425 macro-F1**, confirming the expected advantage of a nonlinear model on a non-linearly-separable, high-dimensional classification problem. The hyperparameter sweep showed activation-function/optimizer compatibility mattered far more than architecture depth or batch size for this dataset and scale.

## 20 References

1. T. E. de Campos, B. R. Babu and M. Varma, “Character Recognition in Natural Images,” *VISAPP*, 2009 (Chars74K dataset).
2. F. Rosenblatt, “The Perceptron: A Probabilistic Model for Information Storage and Organization in the Brain,” *Psychological Review*, 1958.

3. D. E. Rumelhart, G. E. Hinton, R. J. Williams, “Learning Representations by Back-Propagating Errors,” *Nature*, 1986.
4. D. P. Kingma, J. Ba, “Adam: A Method for Stochastic Optimization,” *ICLR*, 2015.
5. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” *JMLR*, 2011.